Matching in Bipartite Graphs (1)

Peerapon S.

CPE 231 Algorithms

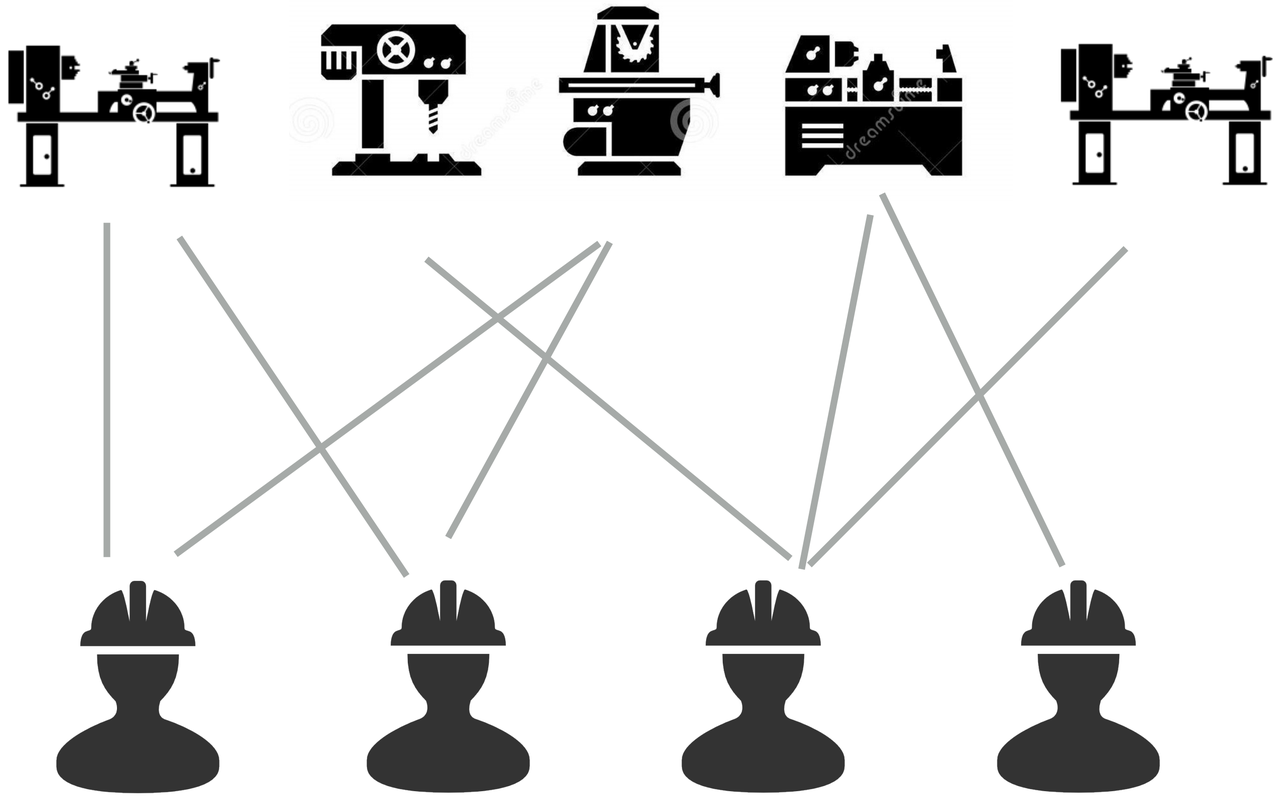
Topics

Matching in bipartite graph and augmenting path

Maximum (cardinality) matching

Maximum matching with ranking → stable matching.

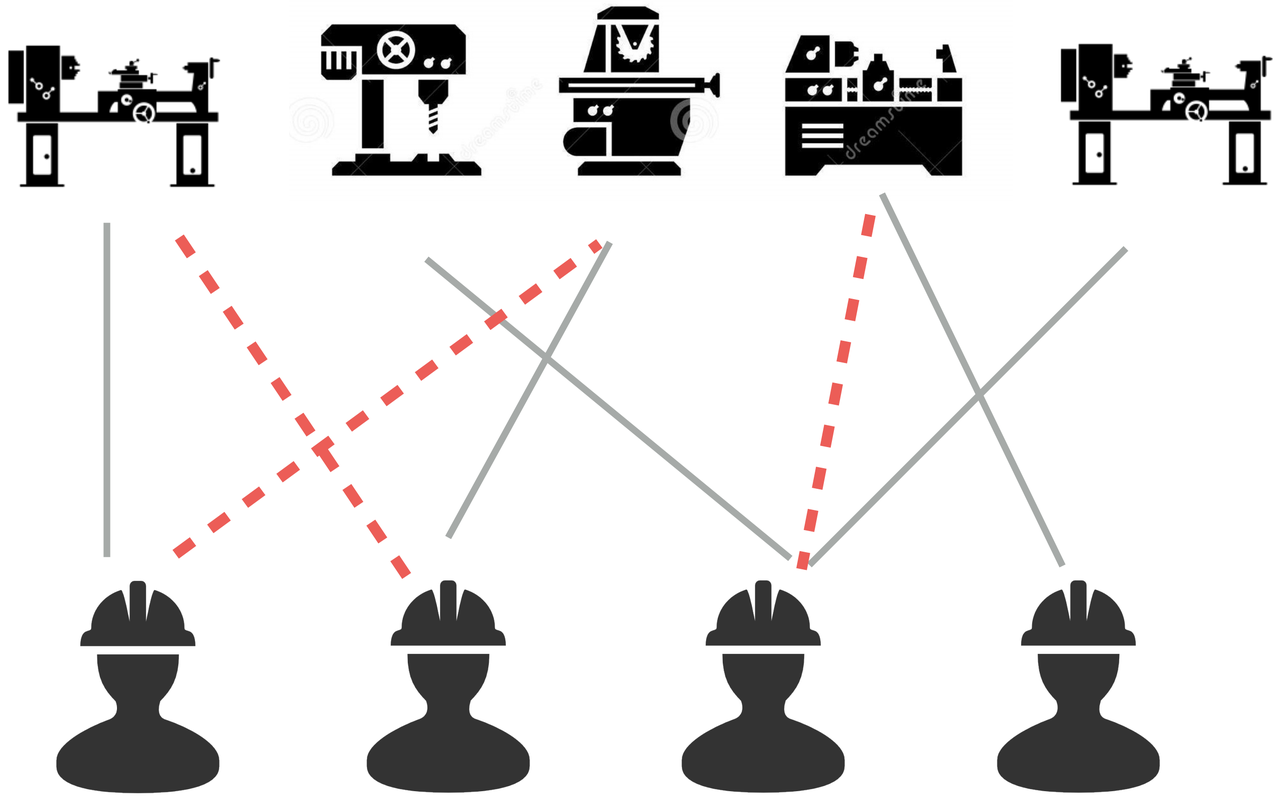
Scenario



n machines
(or holidays,
timeslots)

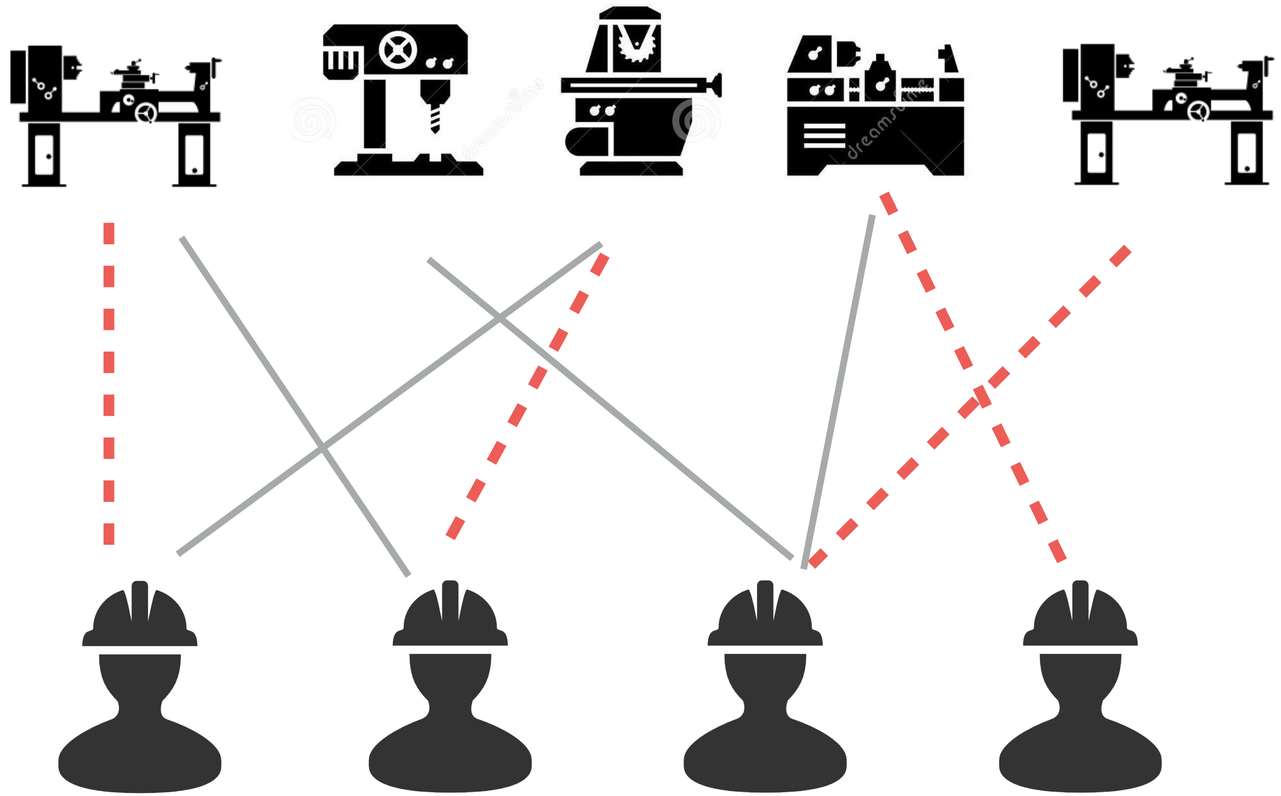
m workers
(or doctors,
jobs)

Not maximum matching. ⇏ This matching is maximal.
But not maximum.



Maximum matching (4 pairs)

This matching is maximal and also maximum.



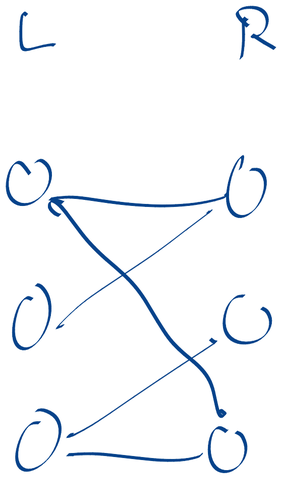
Two Partition

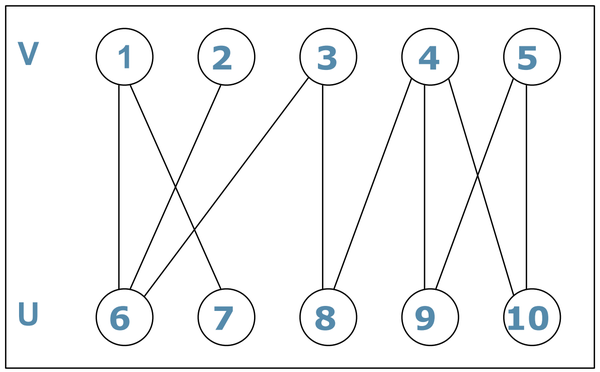
Bipartite Graph $G = (V \cup U, E)$ or $G = (L \cup R, E)$

Basic properties

- Vertices can be partitioned into two disjoint sets V and U, not necessarily of the same size, and
- Every edge $e \in E$ connects a vertex in V to a vertex in U.

Complete bipartite graph if two sets fully connect.





Machines

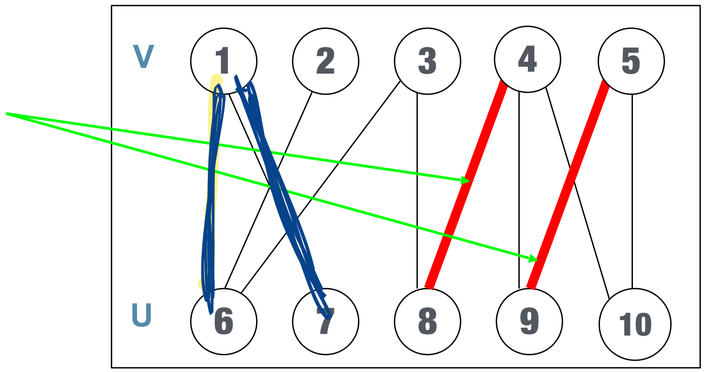
Workers

Matching in a Bipartite Graph

Def: Matching = A subset of edges $M \subseteq E$ where no two edges share a vertex.

Matching

$M = \{(4,8), (5,9)\}$



Is {(1,6),(1,7)} a matching?

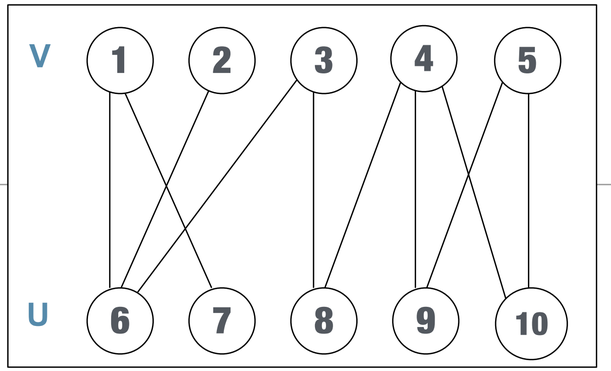
NO!

because they share vertex 1.

Terminology

Matching M is maximal.

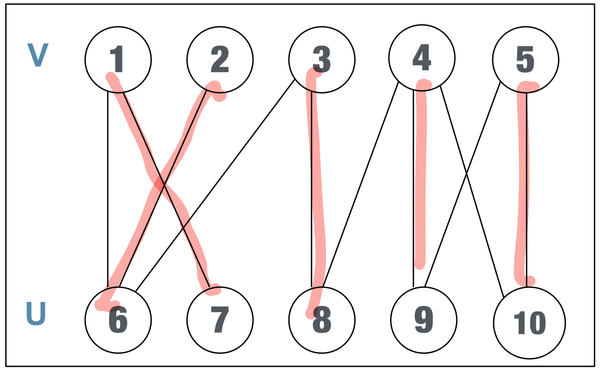
Maximal matching - No additional edge can be added to M.



Matching M is maximum

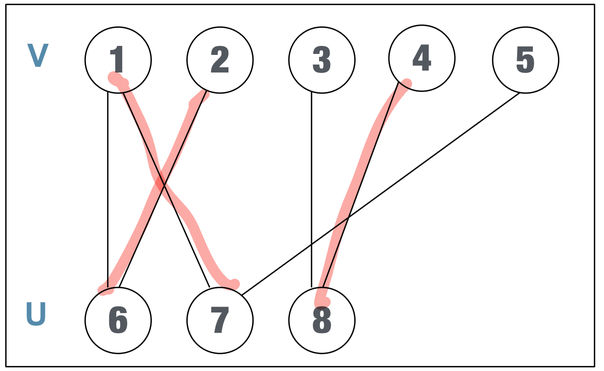
Maximum matching

- Matching with the largest number of edges
- Always exists in bipartite graph but not necessarily unique

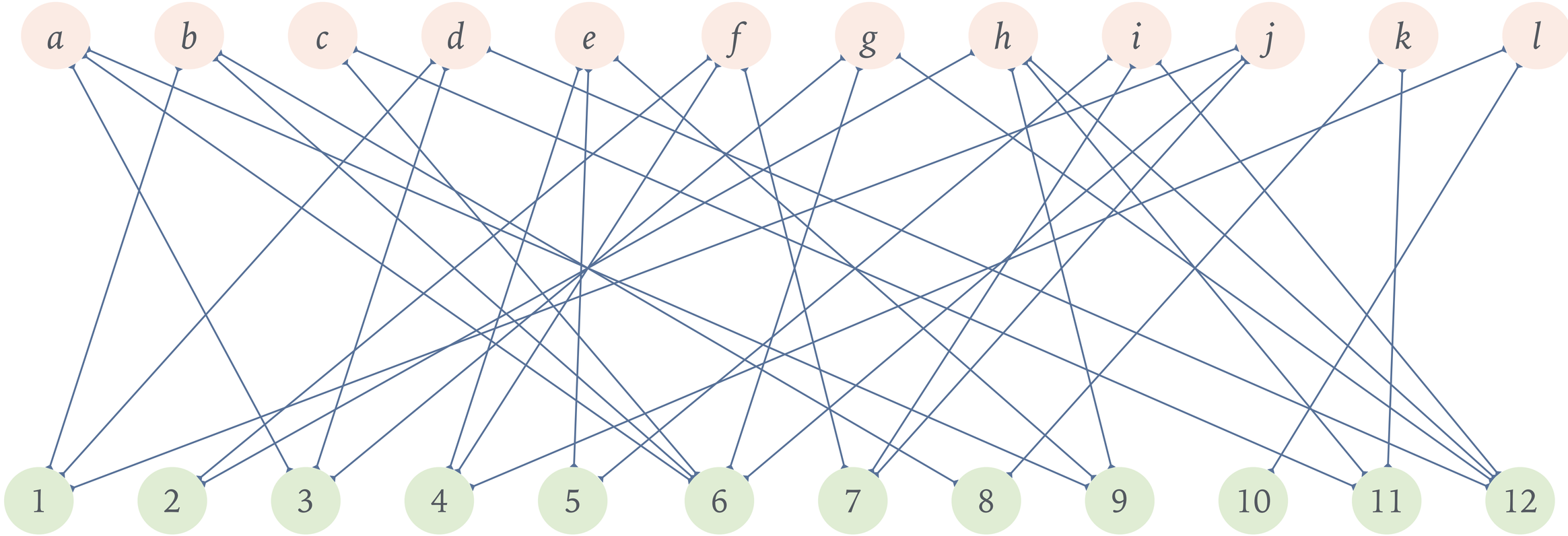


two sides are equal, and all the vertices are matched.

Called ***Perfect matching*** when $|M| = |U| = |V|$



Called ***Complete matching*** when $|U| \leq |V|$ and $|M| = |U|$



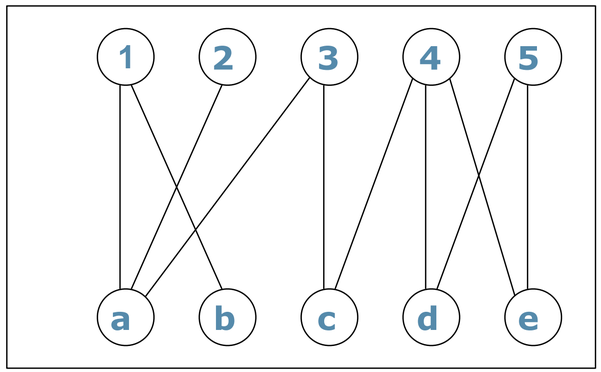
What is the maximum matching for this bipartite graph ?

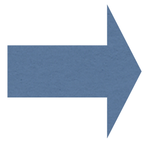
Reduction of Maximum Matching to Max-flow Problem

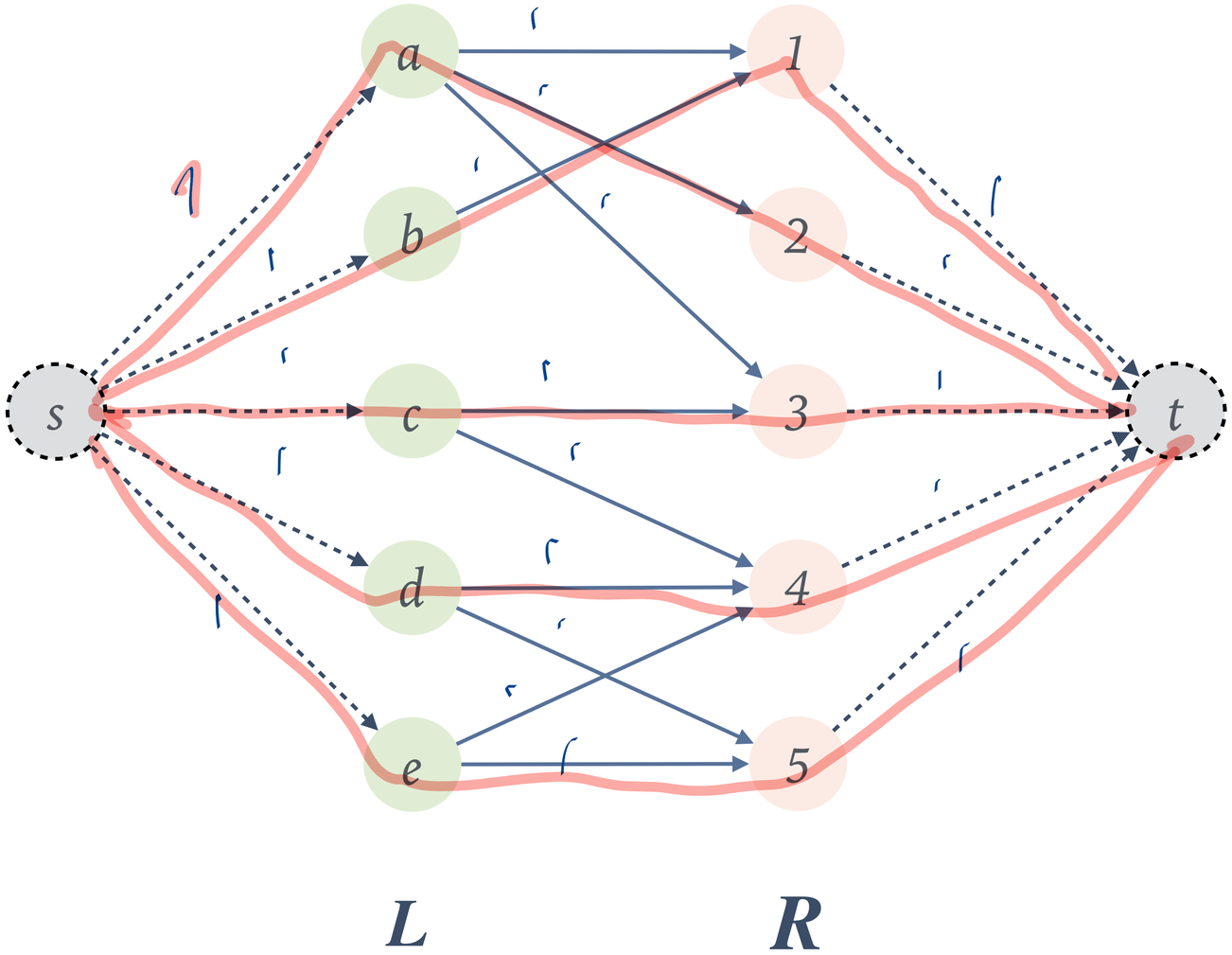
max matching $M = \{(a,2), (b,1), (c,3), (d,4), (e,5)\}$

Flow network G', with $c(e') = 1, \forall e' \in E'$

Graph G



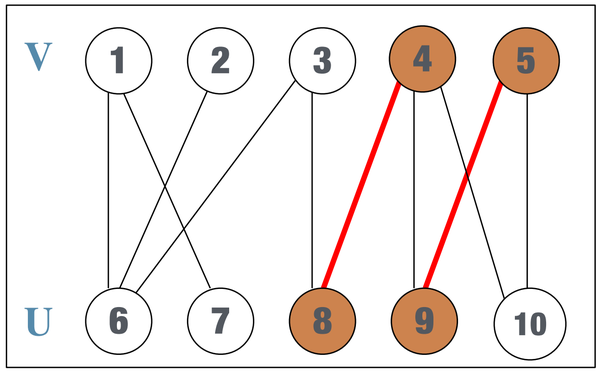




Edmonds-Karp algorithm $O(|V||E|^2)$

Matched Vertex and Free Vertex

$G = (U \cup V, E)$



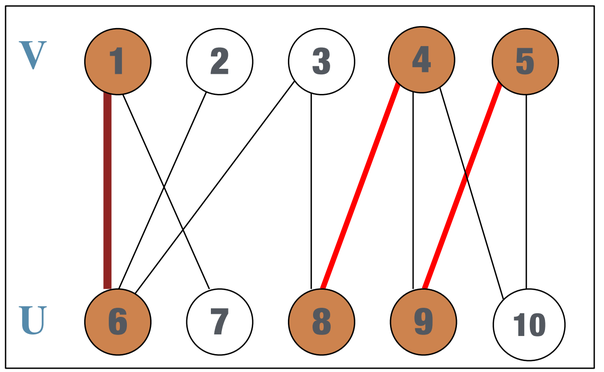
$\boldsymbol{M}$ = {(4,8), (5,9)}

Which vertices are matched ? 4, 8, 5, 9

Which vertices are free ? 1, 2, 3, 6, 7, 10

For a given matching $\boldsymbol{M}$, a vertex in G is either *free* or *matched*

If there are free vertices, then $\boldsymbol{M}$ might be improved.



M-Augmenting Path

M-Augmenting path is a path that:

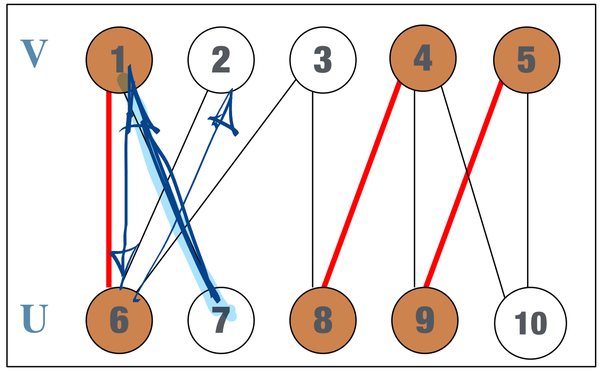
(i) Starts from a free vertex in V and ends at a free vertex in U, or vice versa, and

(ii) Alternates between edges in M^c and M.

↑ edges not in matching.

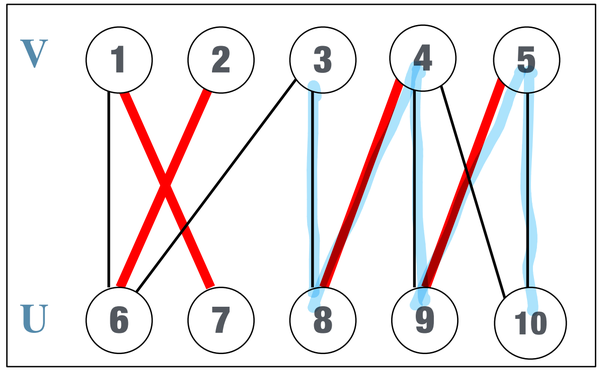
From the matching below, what are M-augmenting paths?

M - Augmenting path = {(7,1),(1,6),(6,2)}



$M = \{ (1,6), (4,8), (5,9) \}$

Self Test: Augmenting Path



Symmetric Difference

For two sets X and Y, its symmetric difference is defined as

$$X \oplus Y = (X \cup Y) - (X \cap Y)$$

Ex: $X = \{1,2,3,4\}$ and $Y = \{3,4,5,6\}$, What is $X \oplus Y$?

$X \cup Y = \{1, 2, 3, 4, 3, 4, 5, 6\}$

$X \cap Y = \{3, 4\}$

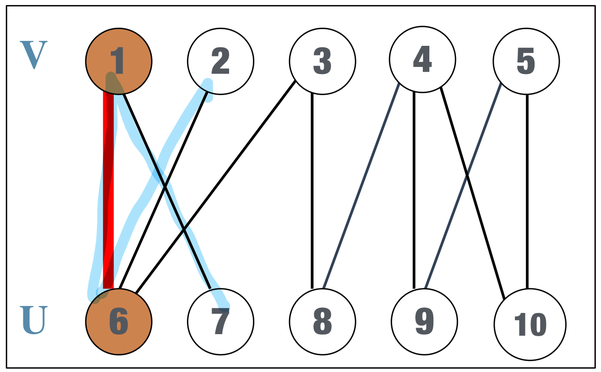
$\Rightarrow X \oplus Y = \{1, 2, 5, 6\}$

How does an augmenting path improving the matching ?

Let M be a matching and P be an M-augmenting path.

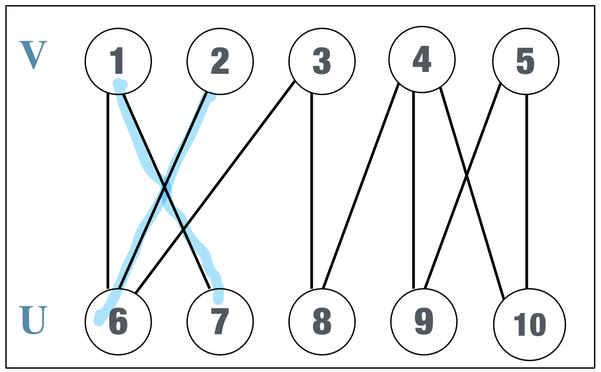
$M = \{(1,6)\}$

$P = \{(2,6), (6,1), (1,7)\}$



$M \oplus P = (M \cup P) - (M \cap P)$

$M' = \{(1,7), (2,6)\}$



The size of the matching is improved by one !

Let M be a matching

Let $P_1, P_2, \ldots, P_k$ be vertex-disjoint M-augmenting paths, and $P = \{P_1 \cup P_2 \cup \cdots P_k\}$.

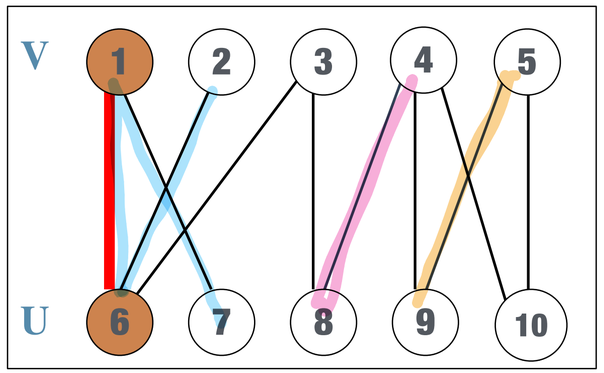
$M = \{(1,6)\}$

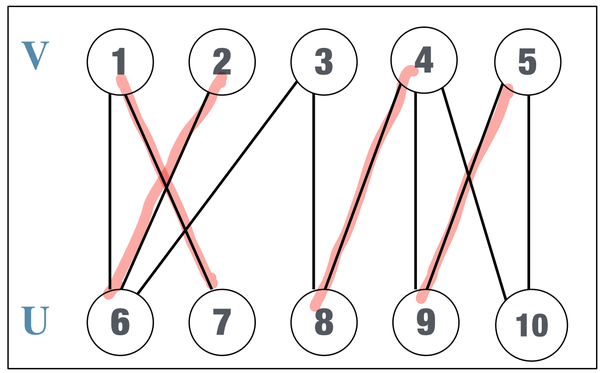
$P_1 = \{(2,6), (6,1), (1,7)\}$,

$P_2 = \{(4,8)\}$

$P_3 = \{(5,9)\}$

$$M \oplus P = (M \cup P) - (M \cap P)$$

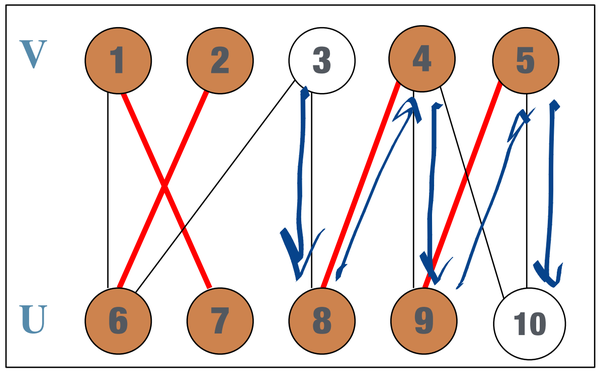




k vertex-disjoint augmenting paths $\Rightarrow$ Matching size is improved by k!

Theorem: A matching M in G is maximum if and only if there exists no M-augmenting path.

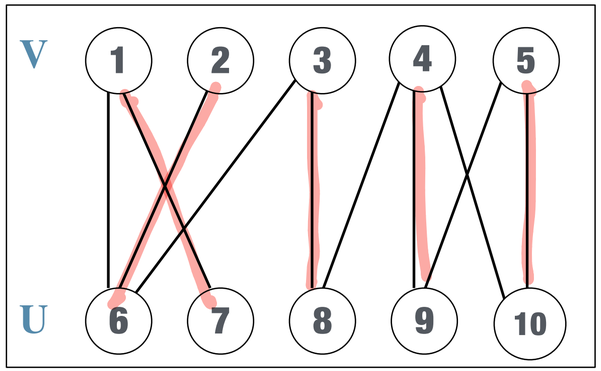
M



Augmenting path P

$= \{(3,8),(8,4),(4,9),(9,5),$
$(5,10)\}$

$M \oplus P$



No Augmenting path can be found.

$\therefore$ The matching is maximum.

Hopcroft-Karp Algorithm

Apply BFS+DFS to find augmenting paths and solves the problem in $O(\sqrt{|V|}\,|E|)$.

Input: Undirected bipartite graph $G = (V, E)$, $V = L \cup R$

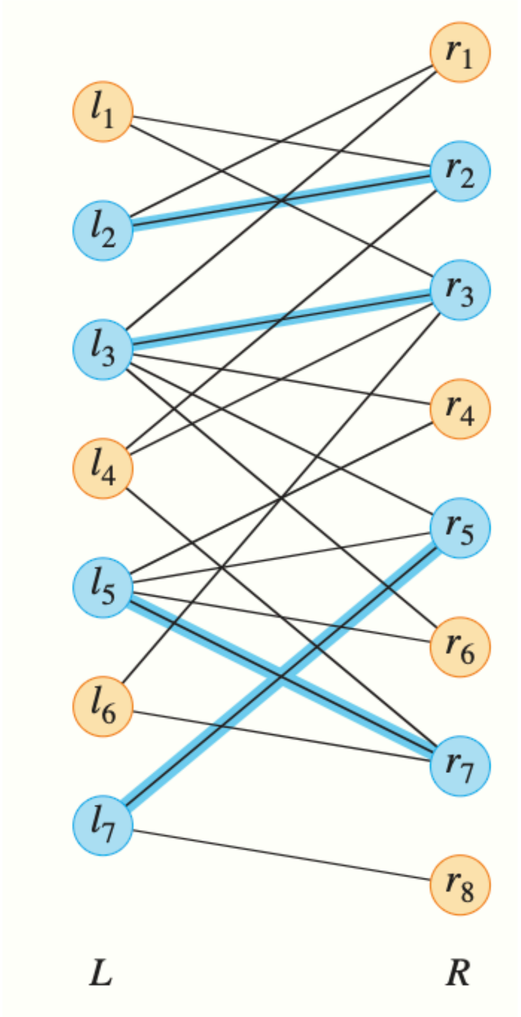
Line 2 iterates at most $2\sqrt{|V|}$ times

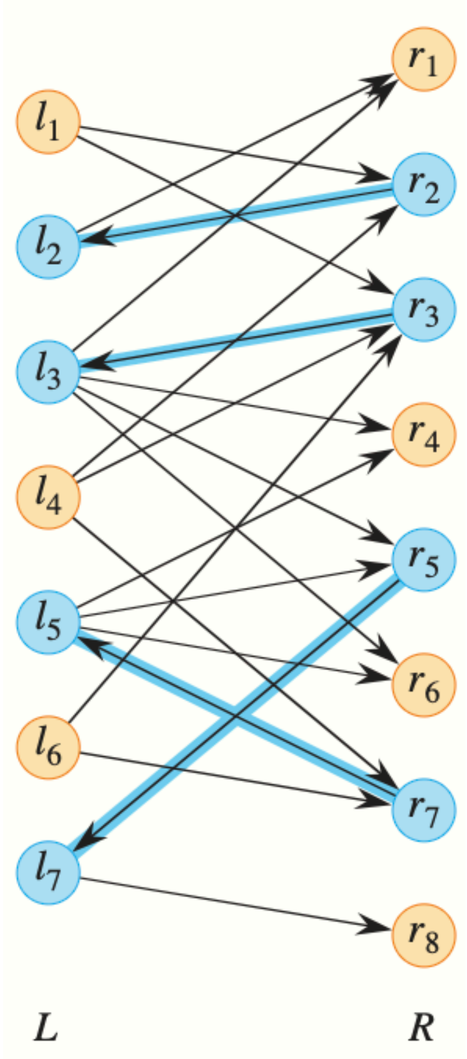
Line 3 is $O(|E|)$ and consists of three phases

```
HOPCROFT-KARP(G)
1  M = ∅
2  repeat
3      let P = {P_1, P_2, ..., P_k} be a maximal set of vertex-disjoint
           shortest M-augmenting paths
4      M = M ⊕ (P_1 ∪ P_2 ∪ ··· ∪ P_k)
5  until P == ∅
6  return M
```

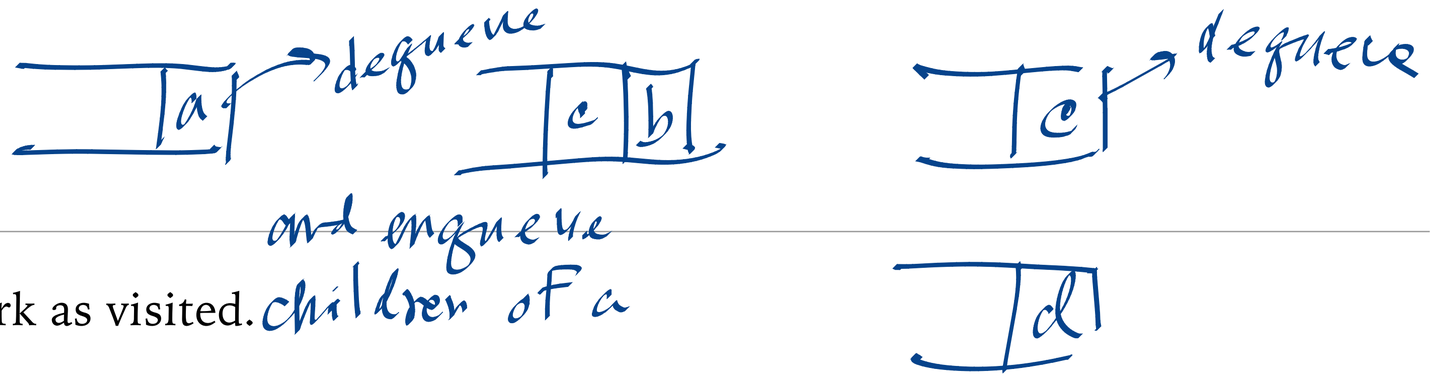
Phase 1: Create a Digraph G_M from G

$$G_M = (V_M, E_M)$$





Review: BFS using Queue

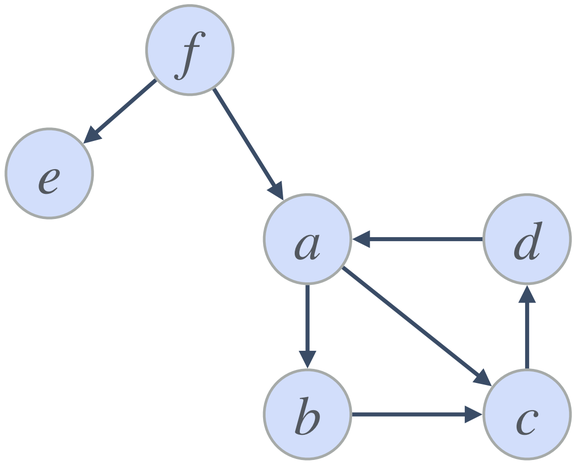


Initialization: Enqueue a starting vertex and mark as visited.

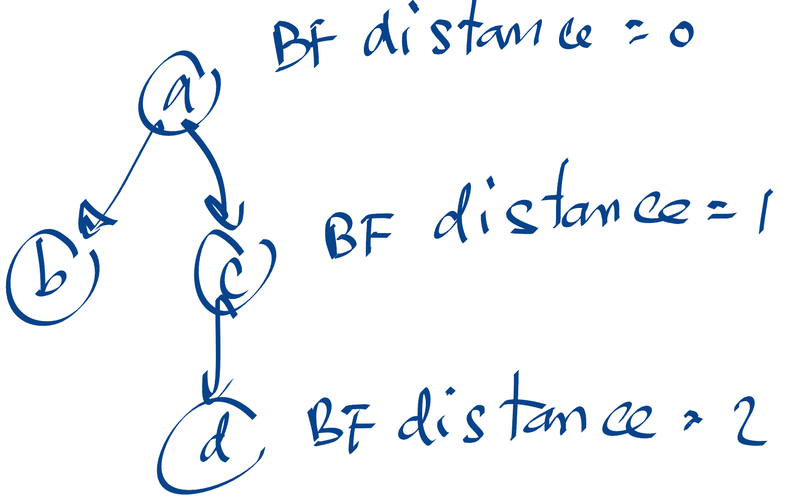
While the queue is not empty:

- Dequeue the front node v.
- Enqueue all unvisited neighbors of v, mark as visited, and note v as their predecessor.

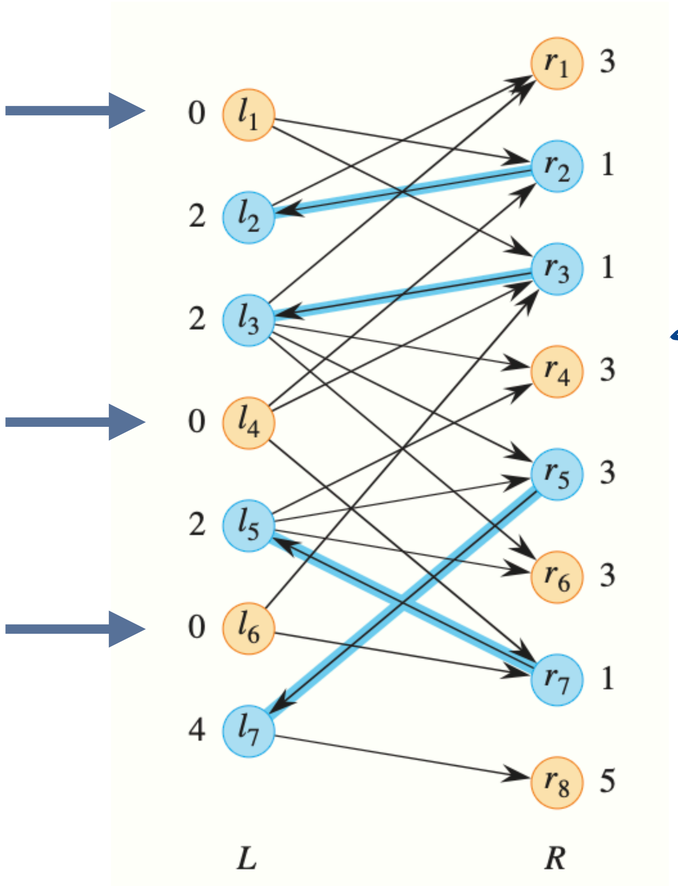
BFS distance = # edges from its root in the BFS tree.



BFS Forest



Phase 2 : Run BFS on G_M to obtain DAG H

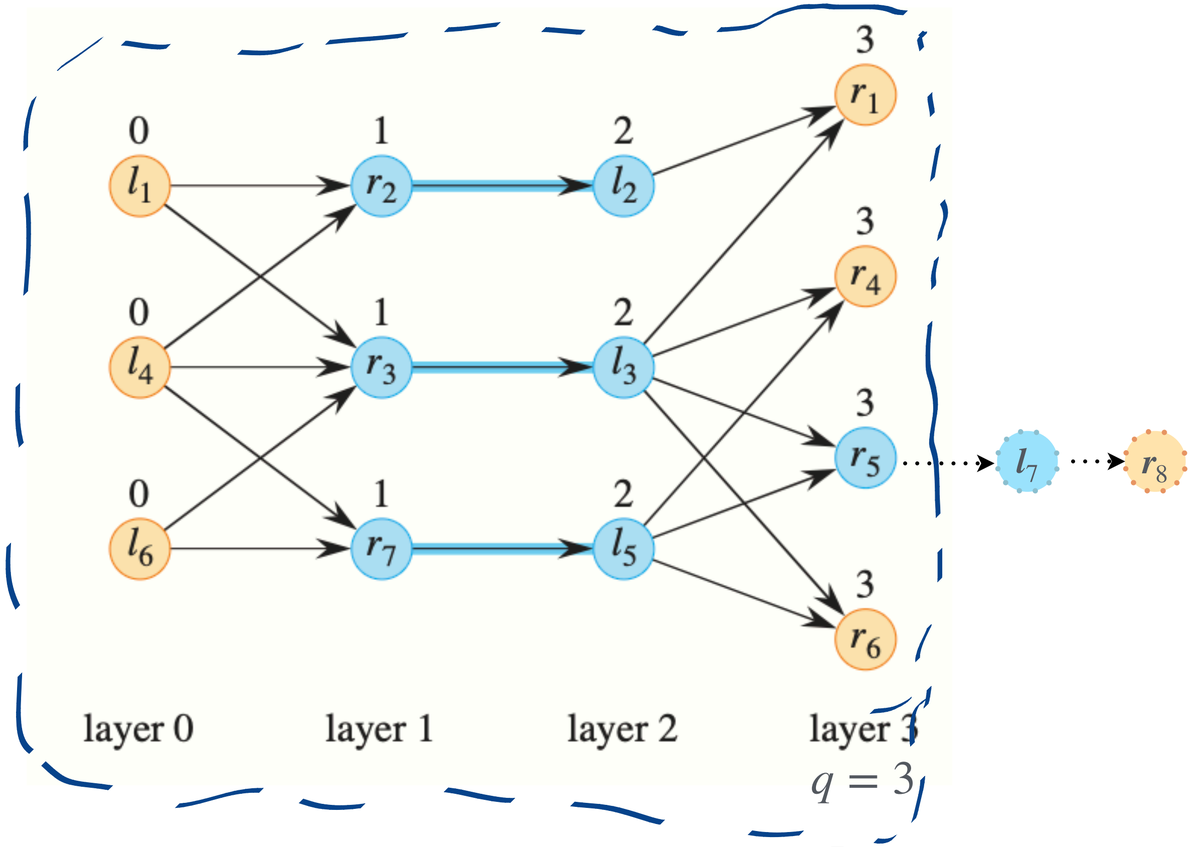


Initial

l_6 | l_4 | l_1

Free vertices on L

DAG $H = (V_H, E_H)$



Labels = minimum breadth-first (BF) distance from any free vertex in L (# edges in augmenting path)

This step takes $O(|V_M| + |E_M|) = O(|E_M|) = O(|E|)$

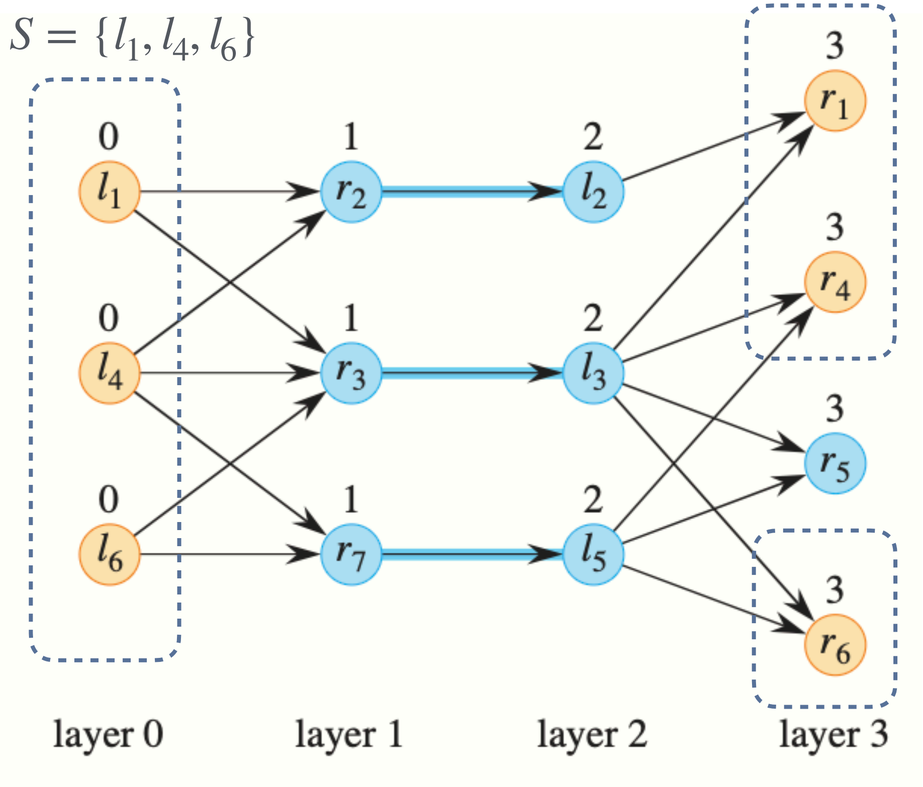
Start BFS by first enqueuing all free vertices in L and put vertices in layers.

A vertex may have many predecessors.

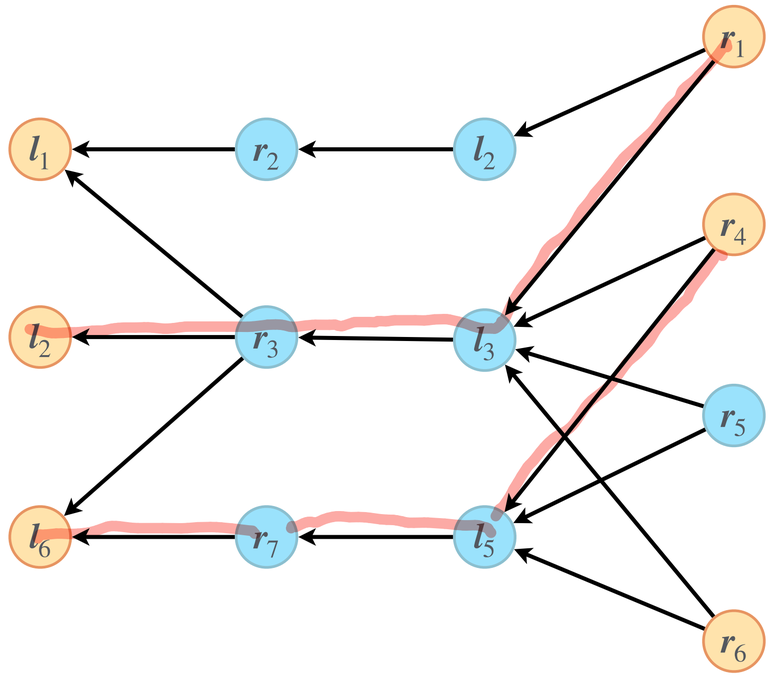
Once all the BFS are done, derive graph H so that its last layer contains vertices in R with distance $q \geq 1$, where q is the smallest BF distance in G_M of any free vertex in R.

Phase 3 : Identify maximal set of vertex-disjoint shortest M-augmenting paths

DAG $H = (V_H, E_H)$ $F = \{r_1, r_4, r_6\}$



H^T



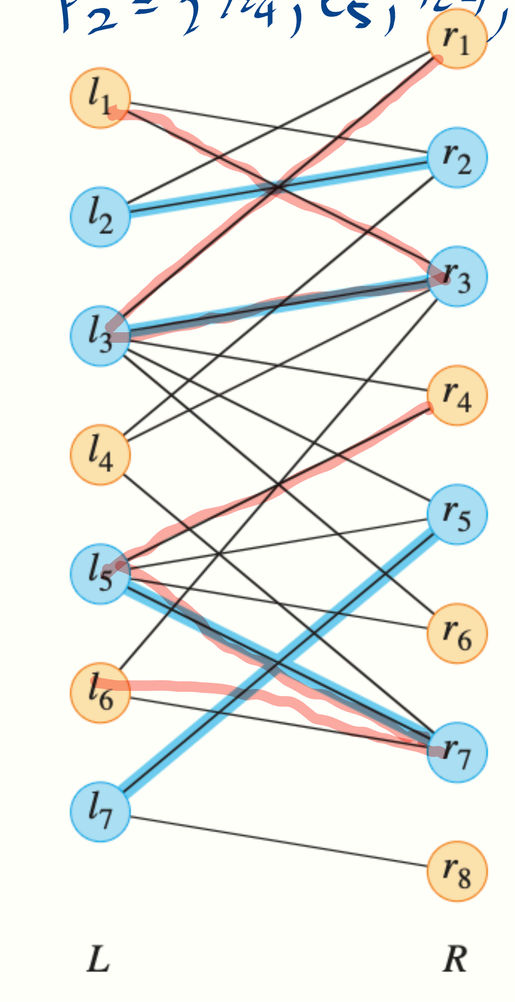
- Run DFS on H^T from each $r \in F$ until reaching $\exists l \in S$.
- Vertices of previous paths automatically excluded.
- Set of paths found is ***maximal***, not maximum.

This step takes $O(|V_H| + |E_H|) = O(|E|)$

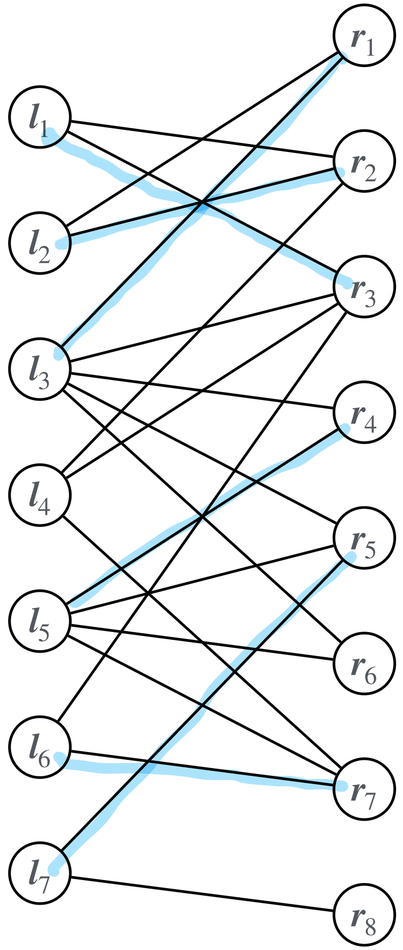
How is the current matching improved ?

$P_1 = \{r_1, l_3, r_3, l_5\}$

$P_2 = \{r_4, l_6, r_7, l_7\}$



2 more edges added to the previous matching.



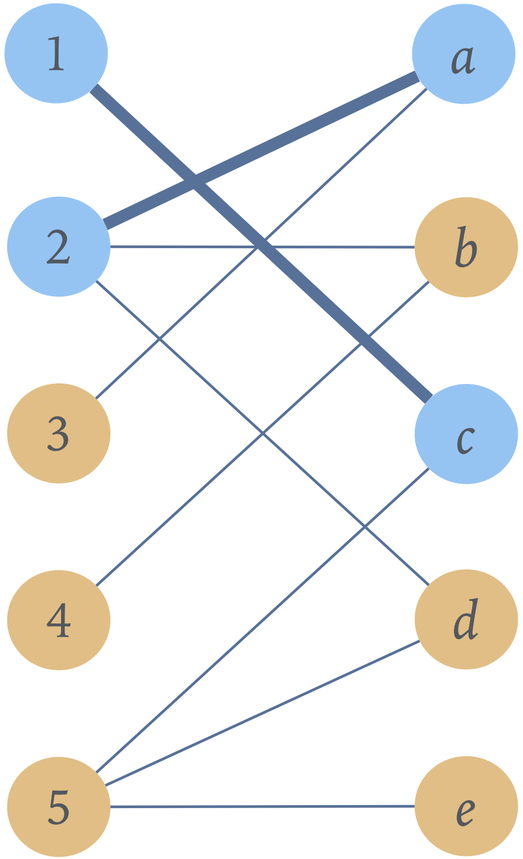
In-class Assignment L6-1 (Group of 3 or less): Hopcroft-Karp Algorithm

(25 minutes) Given a current matching $M = \{(1,c),(2,a)\}$ on the given graph G, work out Hopcroft-Karp algorithm to determine the maximum matching in G.

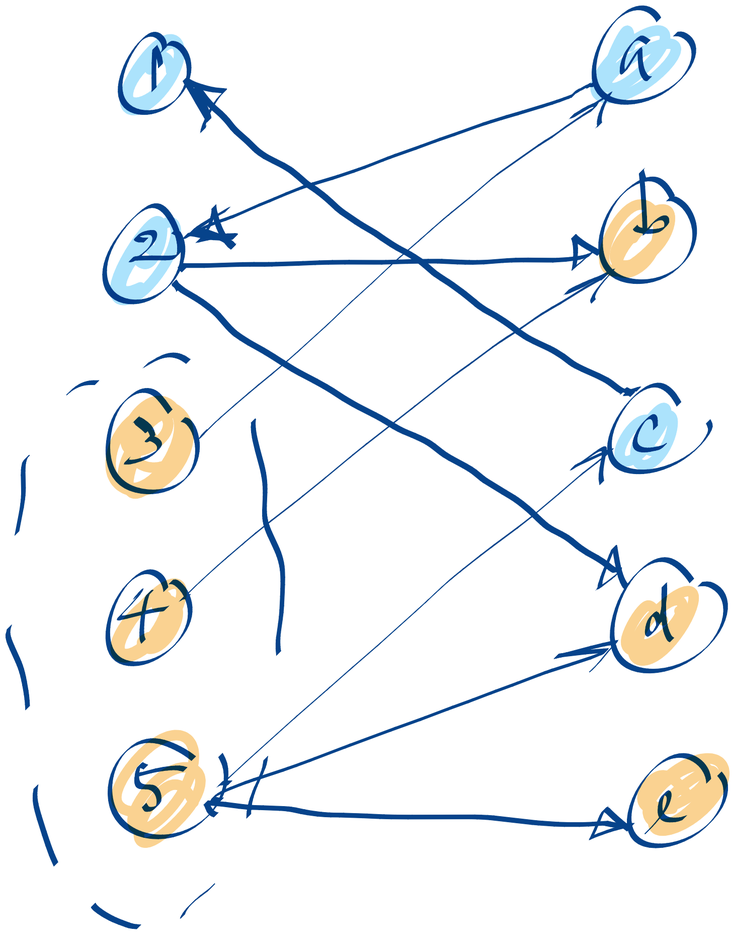
In each iteration of lines 3 and 4,

- Draw the digraph G_M
- Draw in DAG H obtained from running BFS on G_M
- Identify a maximal set of vertex-disjoint shortest M-augmenting paths from DAG H
- Show how the current matching M is improved in G.

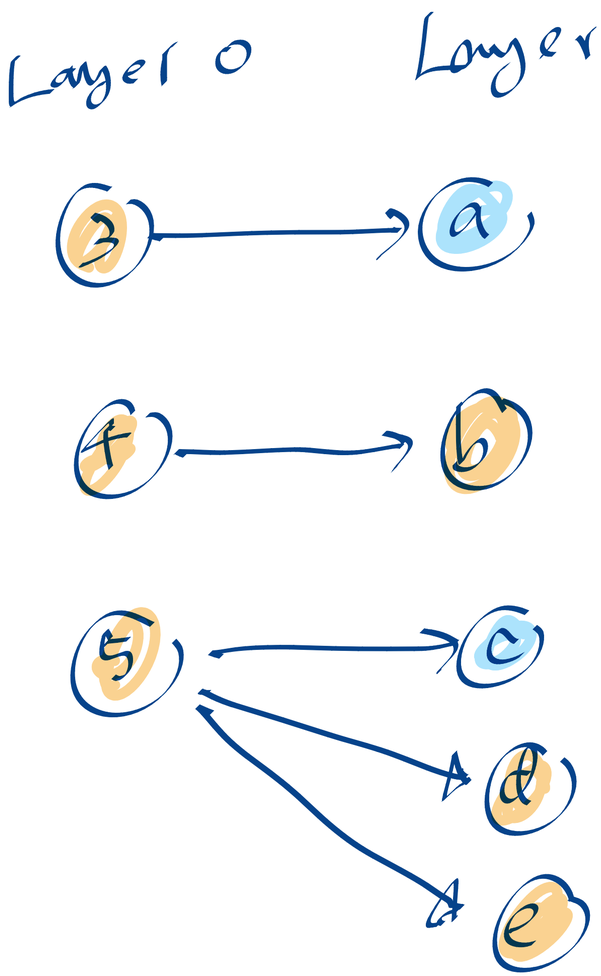
1st iteration

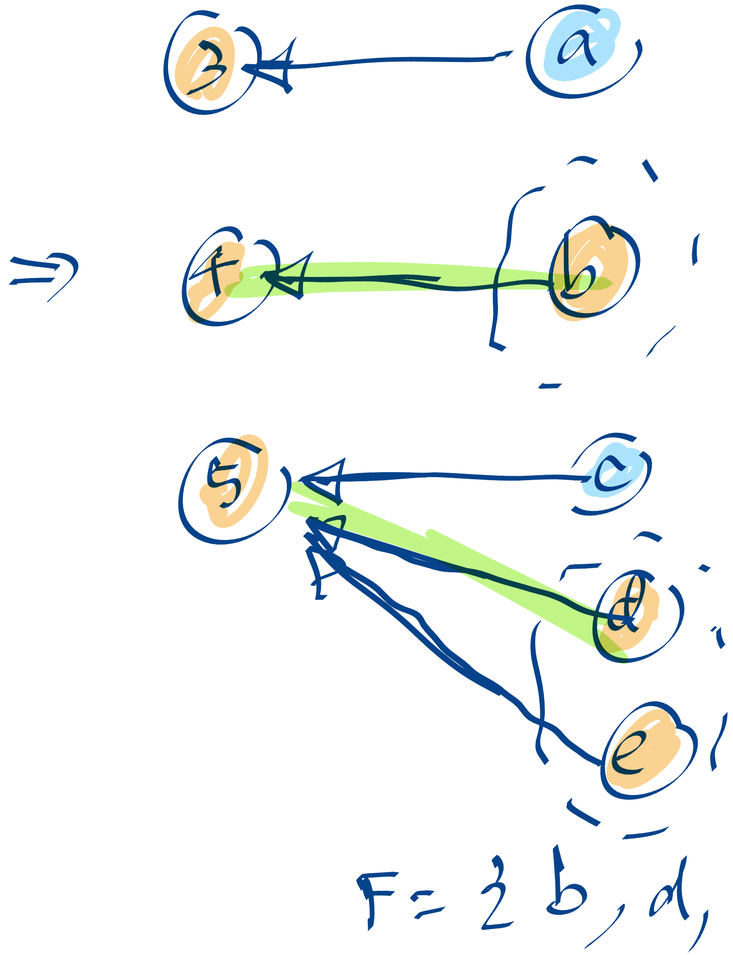


Digraph G_M



DAG $H = (V_H, E_H)$



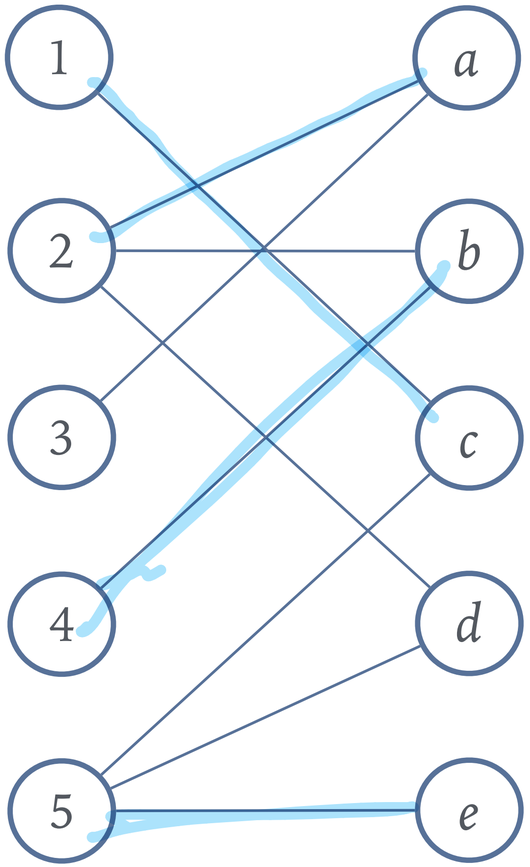


Maximal set of vertex-disjoint shortest M-augmenting paths

{ (b, 4) }
{ (d, 5) }

start DFs from b, d, e one by one

2nd iteration



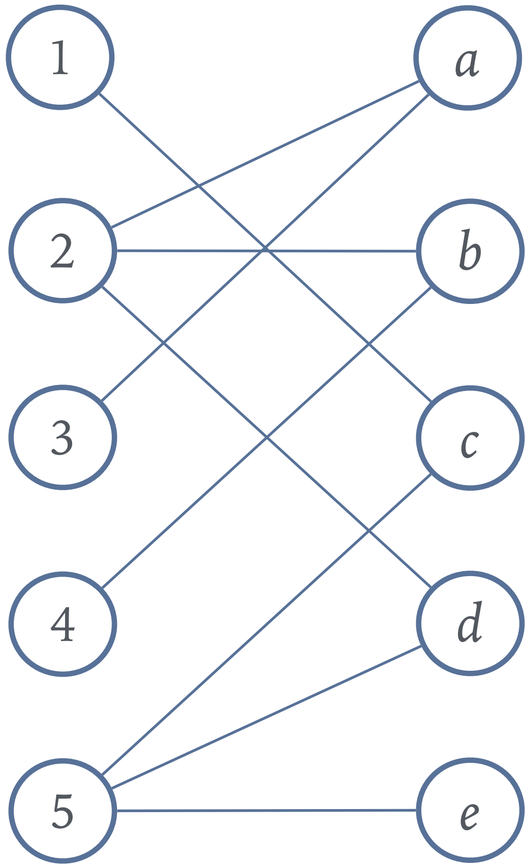
G_M

?

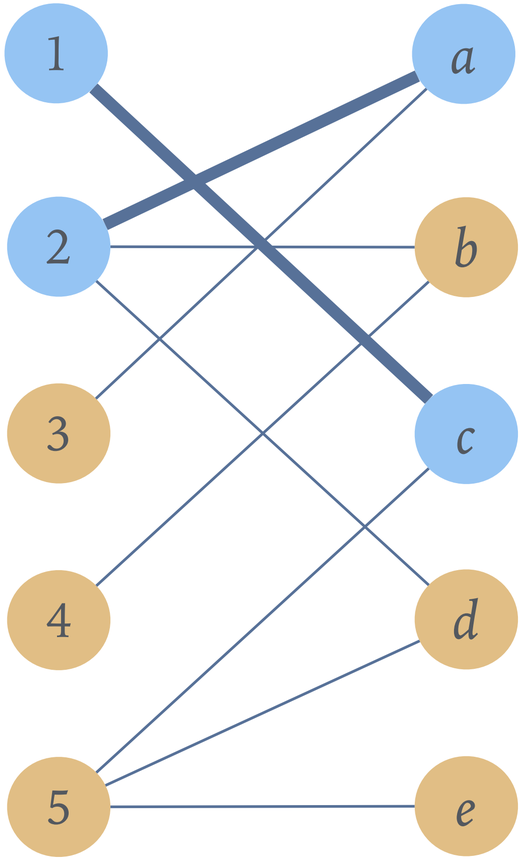
DAG $H = (V_H, E_H)$

Maximal set of vertex-disjoint shortest M-augmenting paths

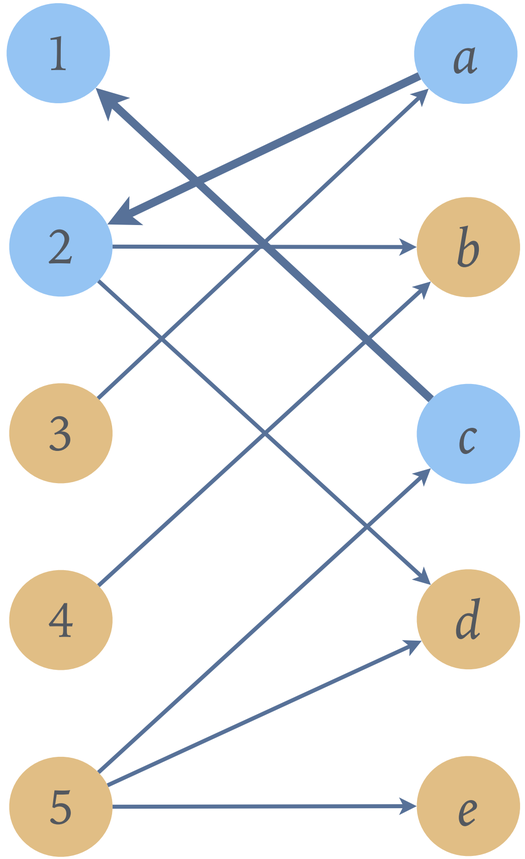
3rd iteration



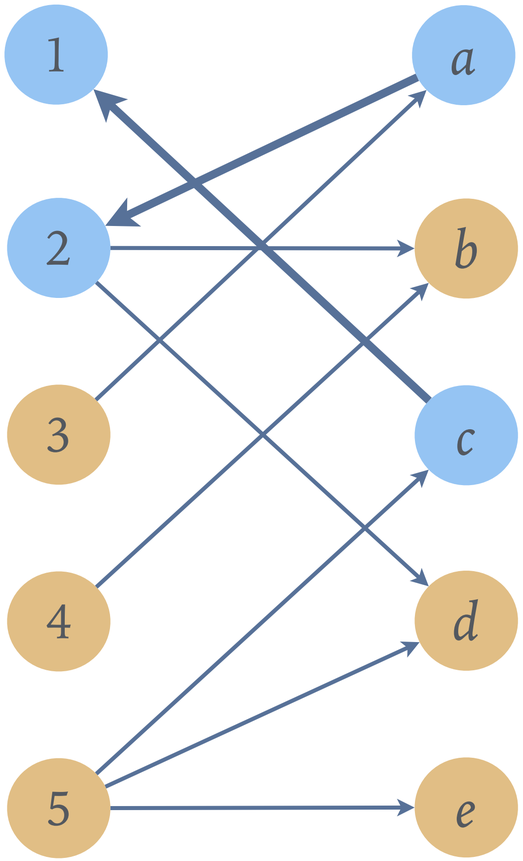
1st iteration



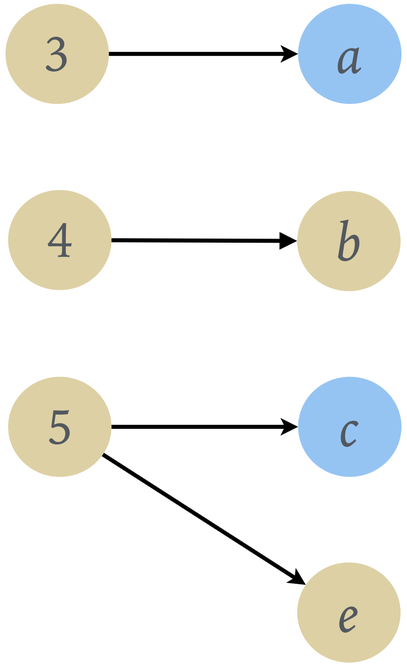
Digraph G_M



Solution

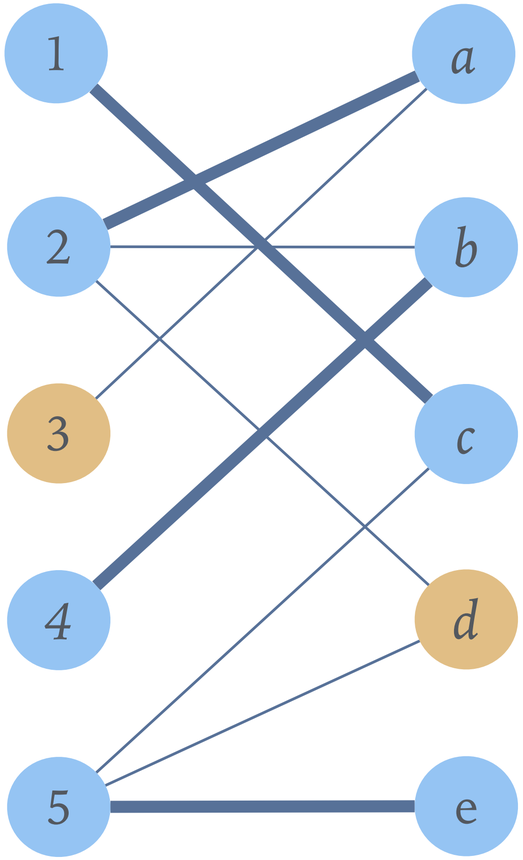


DAG $H = (V_H, E_H)$

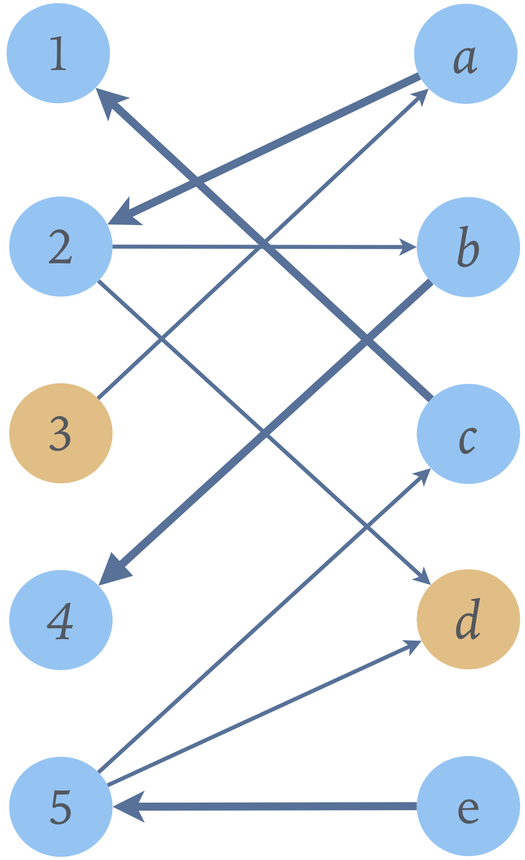


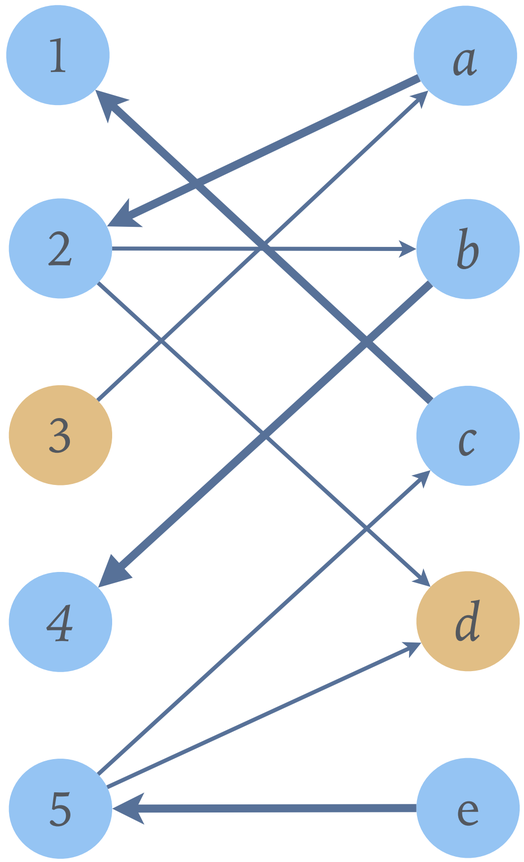
Maximal set of vertex-disjoint
shortest M-augmenting paths: P1 = {(4,b)}, P2={(5,e)}

2nd iteration

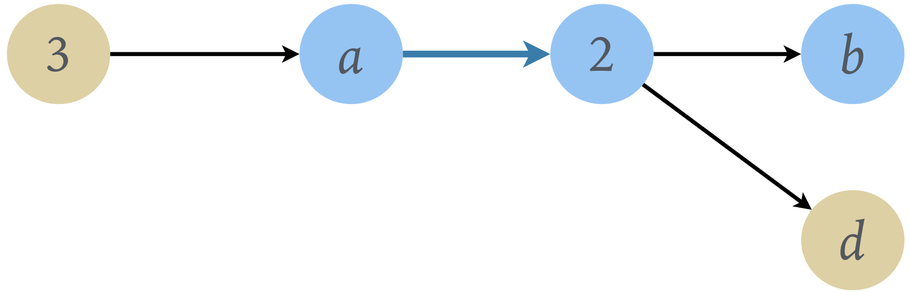


Digraph G_M



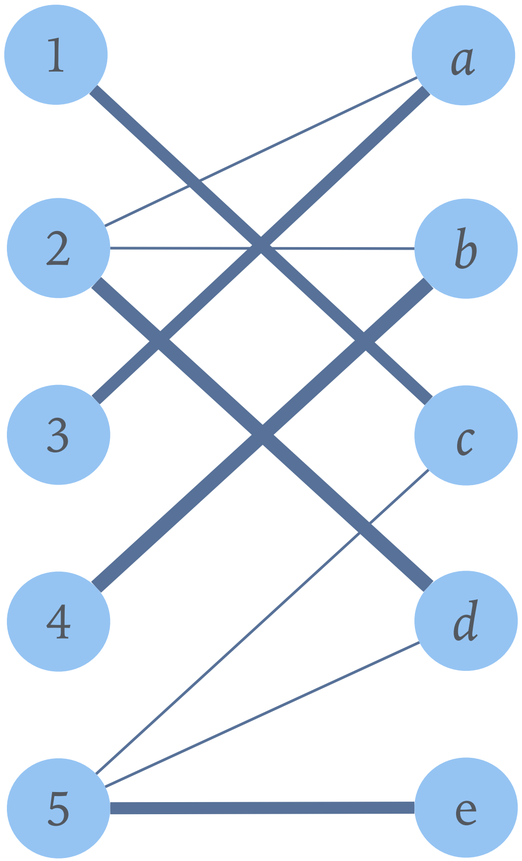


DAG $H = (V_H, E_H)$



Maximal set of vertex-disjoint
shortest M-augmenting paths: P1={(3,a), (5,e)}

3rd iteration



How 47,000 doctors get placed on a single day

THE STORY

Every spring, tens of thousands of graduating US medical students learn where they will spend residency — all at once, on “Match Day.” Nobody negotiates. An algorithm takes each applicant's ranked list of hospitals and each hospital's ranked list of applicants and returns an assignment that is stable: no student and hospital would both rather abandon their match for each other.

47,000+

applicants matched to residencies in one run (2025 Match) — no haggling

Source: NRMP 2025 Main Match; Gale & Shapley (1962); Nobel Prize 2012.

THE ALGORITHM

Stable matching

Gale–Shapley

Gale and Shapley proved in 1962 that a stable matching always exists, and gave an algorithm that finds one by repeated proposals. The residency match runs a version of it; the same idea assigns children to public schools and, as matching on a graph, chains compatible kidney donors and recipients into transplant cycles.

Roth and Shapley won the 2012 Nobel in economics for this work.

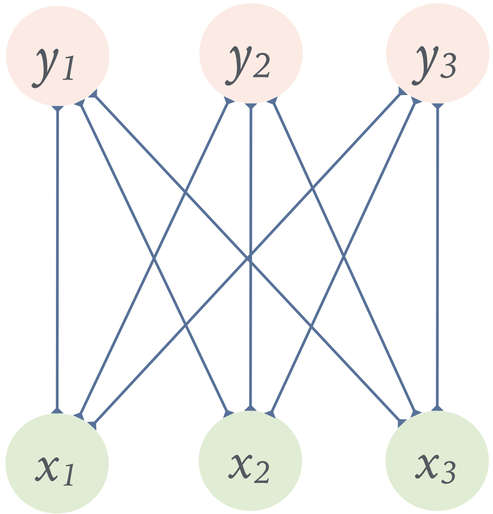
“Stable” is a precise, provable property — and it's why nobody has to negotiate on Match Day.

Maximum Matching with Ranking (Stable matching)

Find desirable maximum matchings among several possibilities.

Set $\mathbf{Y} = \{m_1, m_2, \ldots, m_n\}$ of n men and set $\mathbf{X} = \{w_1, w_2, \ldots, w_n\}$ of n women.

Each man has a ranking list of the women, and each woman has a ranking list of the men (with no ties in these lists).



Ranking Matrix

Men's preferences

```
Bob : {Lea, Ann, Sue}
Jim : {Lea, Sue, Ann}
Tom : {Sue, Lea, Ann}
```

Women's preferences

```
Ann : {Jim, Tom, Bob}
Lea : {Tom, Bob, Jim}
Sue : {Jim, Tom, Bob}
```

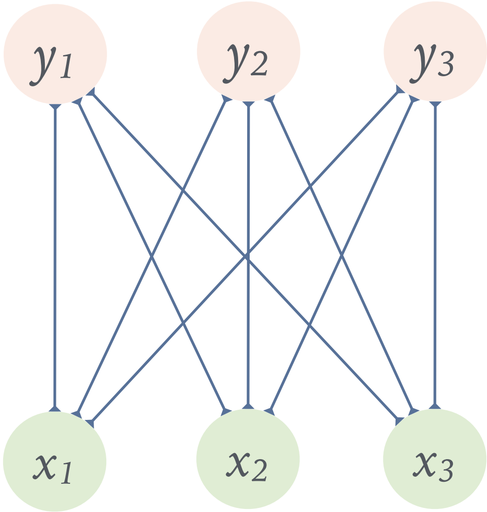
	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Marriage Matching

A ***marriage matching M*** is a set of n pairs $\{(m_i, w_j)\}$.

Ex:

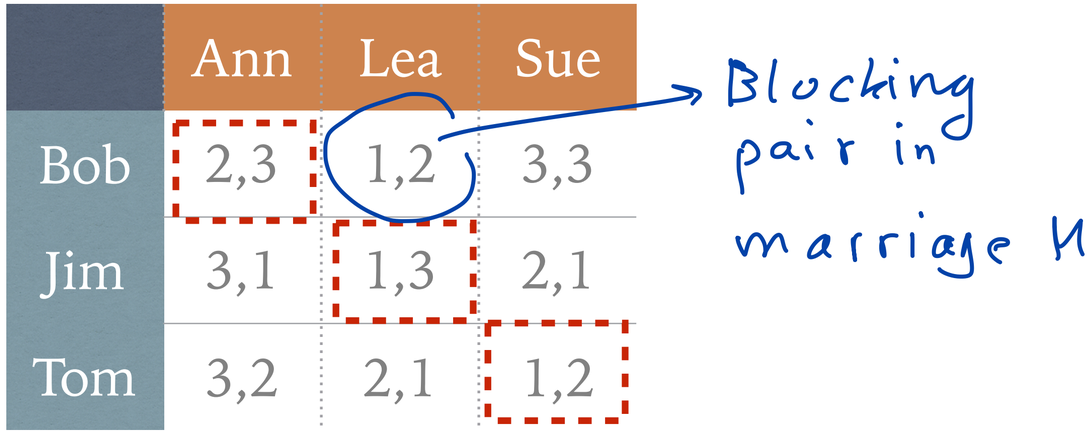
- M = {(Bob, Lea), (Jim, Sue), (Tom, Ann)}
- M = {(Bob, Ann), (Jim, Sue), (Tom, Lea)}



	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Blocking Pair

A pair (m, w) is said to be a ***blocking pair*** in marriage matching $\boldsymbol{M}$ if man m and woman w are not matched in $\boldsymbol{M}$ but prefer each other to their mates in $\boldsymbol{M}$.



Stable Marriage Problem

A marriage matching ***M*** is called ***stable*** if there is no blocking pair; otherwise, it's called ***unstable***.

The stable marriage problem is to find a stable marriage matching for men's and women's given preferences.

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

How many ways to match n men to n women ?

Stable Marriage Algorithm (by Gale-Shapley)

Step 0: Start with all the men and women being free

Step 1: While there are free men, arbitrarily select one of them (m) and do the following:

Proposal m proposes to w, the next woman on his preference list (highest-rank not reject him before)

Response If w is free, match m and w.

If w is not free, she compares m with her current mate m'.

If w prefers m to m', match m and w and make m' free;

Otherwise, w rejects m's proposal, leaving m free

Step 2: Return the set of n matched pairs (perfect matching).

Free men:

Bob, Jim, Tom

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Actions:

Bob proposed to Lea
Lea accepted.

Jim, Tom

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Jim proposed to Lea
Lea rejected.

Jim, Tom

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Jim proposed to Sue
Sue accepted

Free men:

Actions:

Tom

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Tom proposed to Sue.
Sue rejected.

Tom

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Tom proposed to Lea
Lea replaced Bob
with Tom.

Bob

	Ann	Lea	Sue
Bob	2,3	1,2	3,3
Jim	3,1	1,3	2,1
Tom	3,2	2,1	1,2

Bob proposed to Ann
Ann accepted.

The algorithm terminates after no more than n^2 iterations (in Step 1) with a stable marriage output (perfect matching).

An optimal matching is unique for a given set of participant preferences

Matching produced is always ***man-optimal***:

- Each man gets the highest-ranked woman possible on his list under any stable marriage.
- Woman-optimal version is achieving by making women propose to men.

In-Class Assignment L6-2 (Group of Three or less): Stable Marriage Matching

From the following instance of the stable marriage problem, find a stable marriage matching by applying the stable marriage algorithm in its women proposing version. In each iteration, indicate the proposing person and the person rejecting/accepting the proposal, and the matching pairs.

	Ann	Lea	Sue
Bob	1,2	2,3	3,3
Jim	3,1	1,2	2,2
Tom	2,3	1,1	3,1

1. Ann → Jim
2. Lea → Tom
3. Sue → Tom, Tom rejects
4. Sue → Jim, Jim leaves Ann and accepts Sue
5. Ann → Bob

College Admissions Problem

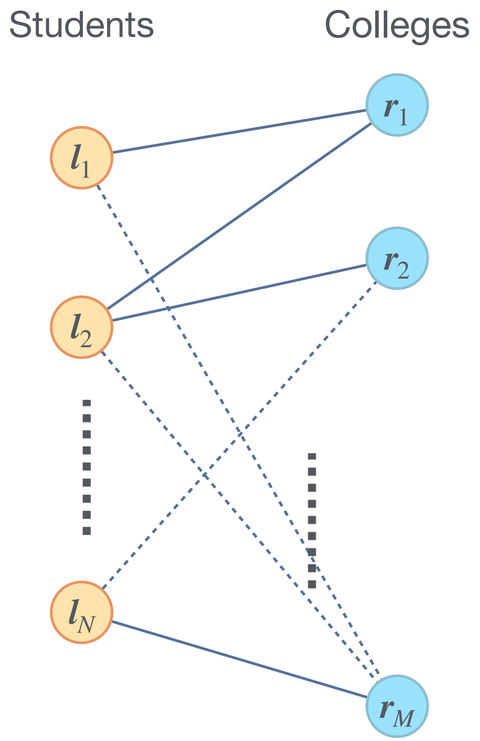
Set of students and colleges

Student side

- Preference list ranking colleges, omitting those never applied.
- Only accept one admission offer

College side

- Preference list ranking applicants, omitting those never applied.
- Have a quota on number of admitted students.



Student-Optimal Deferred Acceptance (SODA) Algorithm

Step 0: Start with all the students being free (not in a waiting list of any college)

Step 1: While there are free students that have not been rejected from college to which they are willing and permitted to apply,

Proposal All free students apply to the college of their choice (highest-rank not rejecting them before)

Response Each college selects the top q from among the new applicants and those already on its waiting list, puts these on its new waiting list, and rejects those exceeding its quota.

Step 2: Each college admits everyone on its waiting list and terminate.

Example

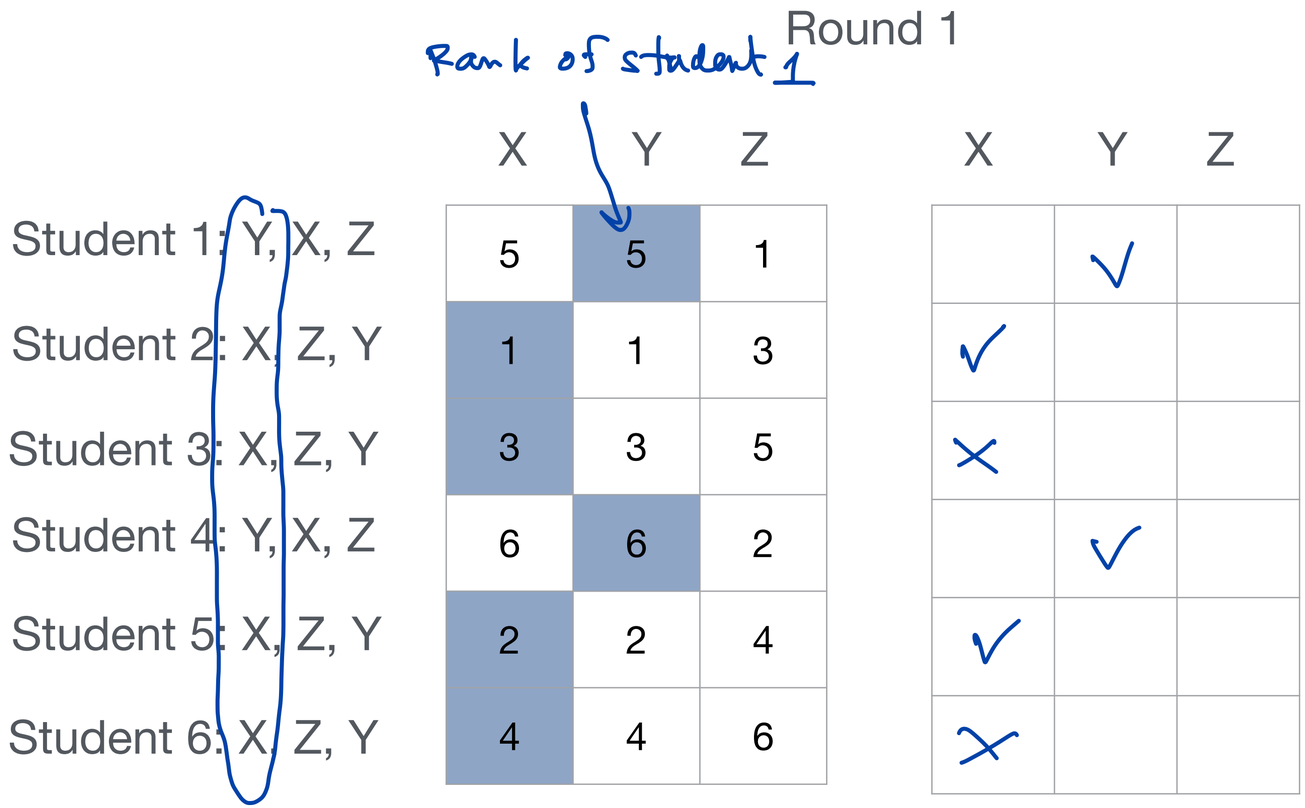
Six students (1-6) and three colleges (X, Y, Z) with quota of two.

Student preferences

- 1,4 : $\{Y, X, Z\}$
- 2,3,5,6: $\{X, Z, Y\}$

College preferences

- X, Y: (2, 5, 3, 6, 1, 4)
- Z: (1, 4, 2, 5, 3, 6)



Example

Six students (1-6) and three colleges (X, Y, Z) with quota of two.

Student preferences

- 1,4 : $\{Y, X, Z\}$
- 2,3,5,6: $\{X, Z, Y\}$

College preferences

- X, Y: (2, 5, 3, 6, 1, 4)
- Z: (1, 4, 2, 5, 3, 6)

Round 2

	X	Y	Z
Student 1: Y, X, Z	5	5	1
Student 2: X, Z, Y	1	1	3
Student 3: X, Z, Y	3	3	5
Student 4: Y, X, Z	6	6	2
Student 5: X, Z, Y	2	2	4
Student 6: X, Z, Y	4	4	6

X	Y	Z
	✓	
✓		
		✓
	✓	
✓		
		✓